

Design and analysis of O(1) Key-Value Storage and Eviction Engine

Mathematical Time Complexity Analysis & Proof

The system implementation requires formal mathematical justification to prove constant time execution. The architecture guarantees deterministic O(1) constant time complexity across all core operations by combining two distinct data structures.

Here is the formal proof of O(1) time complexity broken down by architectural component:

- **Hash Map Lookup:** Proved to operate in O(1) average time via direct array indexing generated from the hash integer.
- **DLL Pointer Rewiring:** Proved to operate in deterministic O(1) time, as locating the node is done instantly via the Hash Map, and detachment requires exactly updating prev->next and next->prev.
- **Positional Updates:** The operation strictly requires no more than four pointer reassignments to move an accessed node to the Head of the list, maintaining mathematical O(1) limits.
- By combining these components, the core API functions achieve absolute constant time execution:

API Operation	Algorithmic Path	Time Complexity
get(key)	Hash Map lookup + DLL pointer rewiring to Head	O(1)
put(key, value)	Hash Map insertion + DLL node creation at Head	O(1)
evict()	DLL Tail detachment + Hash Map key deletion	O(1)

To achieve absolute speed, the system sacrifices a specific amount of physical RAM. The exact byte cost per entry must be calculated and justified using the following formula:

Note:- **The Space-Time Trade-Off: Why It Is Worth It**

Memory Overhead per node = $\text{sizeof}(\text{key}) + \text{sizeof}(\text{value}) + 2 * \text{sizeof}(\text{pointer})$ + HashMap Bucket Overhead

The report must articulate why the tradeoff of utilizing extra RAM for pointer storage is acceptable to achieve computational speed optimization.

Here is the engineering justification for accepting this memory overhead:

- **Eliminating Latency Bottlenecks:** Standard memory reclamation latency bottlenecks inherent to arrays ($O(n)$ shifting) or self-balancing binary search trees ($O(n \log n)$ traversal) are strictly prohibited in the core architecture. Without the two 8-byte pointers on each node, finding the oldest item to evict would require a linear scan of memory, freezing the system during high-load scenarios.
- **Infinite Scalability:** Because of the pointer architecture, the storage engine's operational latency must remain perfectly flat and scale independently of the overall dataset size.
- **Predictable High Throughput:** The system must maintain constant $O(1)$ performance metrics when processing 1,000 requests, 100,000 requests, and 1,000,000 continuous requests under stress testing. Paying a small, predictable memory tax (16 bytes per node) is universally accepted in modern backend architecture to guarantee that millions of user requests never experience random latency spikes during cache evictions.

1. Proof of $O(1)$ Time Complexity for Hash Map Lookups

A standard Hash Map does not search linearly through memory. It calculates the exact memory location using a mathematical function. The proof relies on the assumption of a uniform hashing function and a controlled load factor.

Mathematical Logic:

1. Let k represent the key provided by the user.
2. The hashing function $h(k)$ processes the key and computes a hash integer i such that $0 \leq i < M$, where M is the number of available array buckets. The time to compute $h(k)$ is dependent on the length of the key, but is strictly bounded by a constant time c_1 .
3. Once the index i is calculated, the system accesses the array bucket $\text{Array}[i]$. Memory arrays allow for direct access addressing, meaning the CPU jumps to that physical memory address in a single hardware instruction, taking constant time c_2 .
4. If a hash collision occurs (multiple keys map to index i), the system traverses a small chain. However, as long as the Hash Map resizes itself to maintain a stable load factor

$\alpha = n/M$ the average length of this chain is exactly α . Traversing it takes time proportional to α , bounding this to a constant c_3 .

Conclusion for the Report:

The total time $T(\text{lookup})$ is bounded by $c_1 + c_2 + c_3 \leq C$. Therefore, Hash Map lookup is mathematically proven to operate in $O(1)$ average time via direct array indexing generated from the hash integer.

2. Proof of $O(1)$ Time Complexity for DLL Pointer Rewiring

When an item is accessed in your cache, it must be moved to the front (Head) of the Doubly Linked List (DLL) to mark it as the most recently used. Standard arrays require $O(n)$ time to shift items over, but DLLs avoid this using direct pointer reassignment.

Mathematical Logic:

Let N_x be the node we want to move. It has two pointers: $N_x \rightarrow \text{prev}$ and $N_x \rightarrow \text{next}$

Because the Hash Map stores the exact memory address (pointer) of N_x , locating the node is done instantly via the Hash Map.

Detaching the node from the middle of the list requires exactly two operations:

1. Update the previous node to point to the next node: $(N_x \rightarrow \text{prev}) \rightarrow \text{next} = N_x \rightarrow \text{next}$
Update the next node to point to the previous node: $(N_x \rightarrow \text{next}) \rightarrow \text{prev} = N_x \rightarrow \text{prev}$ This detachment requires exactly updating $\text{prev} \rightarrow \text{next}$ and $\text{next} \rightarrow \text{prev}$. Re-inserting the node at the Head requires updating the Head's previous pointer and N_x 's new next pointer.

Conclusion for the Report:

The operation must strictly require no more than four pointer reassignments, maintaining mathematical $O(1)$ limits. Because the number of reassignments is exactly bounded to four regardless of whether the list has 10 nodes or 1,000,000 nodes, DLL pointer rewiring is deterministic $O(1)$ time.

3. Proof of $O(1)$ Time Complexity for the LRU Eviction Algorithm

The LRU (Least Recently Used) algorithm triggers when the cache is completely full. To maintain constant speed, finding the oldest item, deleting it, and adding the new item must all happen in $O(1)$ time.

Mathematical Logic:

When an API call to put(key, value) forces the cache to exceed its pre-configured integer capacity threshold, the system autonomously triggers the internal evict() module. The algorithmic path is as follows:

1. **Identify Oldest Data (T₁):** Because we maintain a DLL, we always keep a fixed pointer to the Tail of the list. Finding the oldest node is an instant dereference of the Tail pointer. $T_1 = O(1)$.
2. **Delete from DLL (T₂):** The system will immediately un-link the Tail node and reassign the adjacent pointers in exactly constant time operations during an eviction. $T_2 = O(1)$.
3. **Delete from Hash Map (T₃):** Using the key stored inside the Tail node, the system purges it from the Hash Map. As proven in Section 1, deletion in a Hash Map is $O(1)$. $T_3 = O(1)$.
4. **Free Memory (T₄):** The system dynamically frees the memory to prevent memory leaks. Returning a single block of memory to the OS heap is bounded by a constant time limit. $T_4 = O(1)$.
5. **Insert New Data (T₅):** The new key is hashed and inserted into the map ($O(1)$), and a new node is created at the Head of the DLL ($O(1)$). $T_5 = O(1)$.

Conclusion for the Report:

The total time complexity to evaluate the eviction state, remove the oldest data, and insert the new data is calculated as:

$$T_{\text{Total}} = T_1 + T_2 + T_3 + T_4 + T_5$$

$$T_{\text{Total}} = O(1) + O(1) + O(1) + O(1) + O(1) = O(1)$$

By coupling the associative Hash Map ($O(1)$ lookups) with the Doubly Linked List ($O(1)$ positional updates), the system prevents the standard memory reclamation latency bottlenecks of $O(n)$ or $O(n \log n)$. Thus, the entire LRU eviction engine mathematically guarantees deterministic constant time execution.